An Authoritative STM32 Tic-Tac-Toe Game Server with a Thin Python GUI Client

EEE3095S CS Project 2026

Nemukula Vhugala (NMKVHU004) Munyai Umusindisi Cindy (MNYUMU003)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—This paper presents the design, implementation, and empirical verification of an authoritative Tic-Tac-Toe serial game server deployed on an STM32F051 microcontroller paired with a thin Python Tkinter GUI client over an asynchronous serial link at 115 200 baud. The microcontroller operates as the sole referee and single source of truth, enforcing game state, turn alternation, and move legality through a five-stage gatekeeping pipeline before committing any changes to memory. Protocol robustness is ensured through an interrupt-driven circular ring buffer with a 64-byte frame ceiling, silent carriage-return stripping, and delimiter-based resynchronization that returns specific error codes without state corruption. System verification was conducted using both an automated `pytest` test suite and a hardware regression harness driving the physical board over its Virtual COM Port. The system achieved 100% pass rates across all 48 automated unit and integration tests as well as 26 physical hardware assertions, with an average measured round-trip transaction latency of 4.2 ms.**

I. Introduction

Game software normally runs entirely on a PC. The PC decides whether a move is legal, tracks the board, and detects the winner. This design has a serious weakness: a client that sends moves in the wrong order, or claims a square is free when it is not, could corrupt the game. The project brief [1] addresses this by reversing the roles. The STM32 microcontroller is the sole referee. It stores the board, validates every action and returns the authoritative result. The PC client only draws the board and forwards what the player clicks. The central design invariant enforced by the server is that a rejected request leaves the game state strictly unchanged; the board after an invalid move is identical to the board before it.

We implemented **Tic-Tac-Toe** (also called noughts and crosses), the baseline game described in the brief. Tic-Tac-Toe has a small, well-defined state (nine cells, two players, eight winning lines) that fits comfortably in the STM32's memory and can be exhaustively tested in under a minute. For the graphical user interface (GUI) we chose Python's built-in **Tkinter** library [2]. Tkinter requires no additional installation, runs on Windows without extra drivers, and handles user events and screen updates in a single thread using the Tk event loop, which removes the need for explicit threads when waiting for the serial reply.

The system is divided into three layers. The embedded firmware on the STM32 handles all rule enforcement and game logic. A thin serial driver on the PC opens the COM port and exchanges newline-terminated ASCII frames at 115 200 baud. The Tkinter GUI sits on top of the driver and deals only with drawing and user input. This layering means the game rules can be tested independently of the screen by sending raw commands over the serial port, which is exactly how our automated test harness works.

Task allocation. Nemukula Vhugala (Partner A) was responsible for the embedded firmware: the interrupt-driven UART receiver and circular ring buffer (`protocol.c`), the game finite-state machine (FSM) and rule engine (`game.c`), and the hardware integration (button debouncing and LED status indicators, `hardware.c`). Munyai Umusindisi Cindy (Partner B) was responsible for the PC-side software: the serial communication driver (`serial_client.py`), the Tkinter GUI (`gui.py`), and the automated hardware test harness (`test_day3_stm32.py`). Both partners contributed to the protocol specification, system integration, and this report.

Structure. Section II defines the serial protocol, framing, and command set. Section III presents the server FSM, GUI flowchart, and circuit diagram. Section IV explains key firmware and client implementation decisions. Section V details server-side validation and rule enforcement. Section VI covers framing recovery and error handling. Section VII presents automated and hardware test results. Section VIII concludes with limitations and future work. Section IX links the video demonstration, Section X identifies the source repository and commit hash, and the report concludes with formal references.

II. Protocol

A. The Serial Link Carries Plain Text Lines of at Most 64 Bytes

The STM32 communicates with the PC over **USART1** (pins PA9 TX, PA10 RX), which is connected internally to the onboard ST-Link chip. The ST-Link presents a **Virtual COM Port (VCP)** to the host operating system, so no external USB-to-UART adapter is required. The link parameters are 115 200 baud, 8 data bits, no parity, 1 stop bit (8N1), no hardware flow control.

A frame is a sequence of printable ASCII characters terminated by a newline character (`\n`, ASCII 0x0A). Carriage return characters (`\r`) are silently discarded, so Windows-style `\r\n` line endings work without any special handling. No checksum or CRC is used; the link is short and the ST-

TABLE I
FRAMING RULES AND THE SERVER'S RESPONSE TO EACH CONDITION.

Rule	Server behaviour
Terminator	`\n` ends a frame; `\r` is silently discarded
Maximum frame length	64 bytes (excluding delimiter); excess bytes discarded, returns `NACK:BADCMD` upon `\n`
Incomplete frame	Discarded after 500 ms timeout; returns `NACK:BADCMD`
Invalid character	Accumulated until delimiter; rejected by command validator
Checksum	None; short USB-VCP physical layer provides USB CRC
UART errors	Hardware flags cleared in error ISR; receiver re-armed

TABLE II
COMPLETE COMMAND SET. THE SERIAL LINK RECEIVES FRAMES IN ANY STATE, BUT COMMAND EXECUTION DEPENDS STRICTLY ON THE CURRENT FSM STATE. INCOMPATIBLE REQUESTS RETURN `NACK:NOGAME` OR `NACK:BADCMD`.

Request	Valid in	Success response
`CONNECT`	Any state	`OK` (resets session to IDLE)
`NEW`	IDLE, GAME_OVER	`OK:BOARD:.........`
`MOVE:`p`:`i	IN_PROGRESS	`OK:BOARD:`$grid$, `WIN:`p, or `DRAW`
`STATE`	Any state	`BOARD:`$grid$
`RESIGN`	IN_PROGRESS	`OK:GAMEOVER`
`DISCONNECT`	Any state	`OK` (resets session to IDLE)

Link USB transfer provides its own error correction. Table I summarises the framing rules.

B. The Board is Nine Characters, Left to Right, Top to Bottom

The game board is encoded as a string of exactly nine characters. Cell 0 is the top-left corner; cell 8 is the bottom-right corner, reading left to right and top to bottom (row-major order). An empty cell is a full stop (`.`); a cell owned by Player X is the letter `X`; a cell owned by Player O is the letter `O`.

Example. After Player X takes cell 0 and Player O takes cell 4, the board string is `X...O....`, which represents:

```
X | . | .
. | O | .
. | . | .
```

C. Commands Are Synchronous Uppercase ASCII Strings With Single-Line Replies

Table II lists every command the server accepts. All commands are case-sensitive uppercase ASCII. A move command includes the player letter and the target cell index, separated by colons: `MOVE:X:4` means "Player X requests cell 4." The server always responds with exactly one line before accepting the next command.

D. Deterministic Error Codes Map Directly to Specific Rule Violations

Table III lists every error response. The order of checks inside `game_process_move()` matters because a single bad command can fail more than one check. The server applies them in a fixed sequence: (1) syntax and player token, (2) game state, (3) cell index range, (4) turn order, and (5) cell occupancy. This ensures each rejected command yields an unambiguous, predictable error code.

TABLE III
EVERY ERROR RESPONSE AND THE EXACT CONDITION THAT TRIGGERS IT.

Response	Sent when
`NACK:BADCMD`	Unrecognised verb, frame $>$ 64 bytes, 500 ms timeout, invalid player token (not `X` or `O`), or malformed syntax (e.g. missing colon or omitting player)
`NACK:NOGAME`	`MOVE` or `RESIGN` received while in `IDLE` or `GAME_OVER`
`NACK:RANGE`	Move index is not a decimal digit or falls outside the valid range 0 to 8
`NACK:NOTURN`	Claiming player token does not match the active turn (`g_current_player`)
`NACK:OCCUPIED`	The requested cell already contains `X` or `O`

E. Deliberate Protocol Design Choice: Explicit Player Tokens in Move Commands

The reference protocol in the project brief [1] specifies a move command of the form `MOVE:`i (specifying the target cell index only). Under that format, a server enforces alternating turns by assigning each incoming legal move to whichever player currently holds the active turn, advancing its internal turn counter automatically. However, because `MOVE:`i carries no player token, the server has no identity against which to validate the originator of the request; it cannot detect if a specific participant attempts to submit a move out of order.

As permitted by the project brief (which instructs teams to adapt the protocol and highlight departures from the reference), our implementation adopts `MOVE:`p`:`i as a deliberate design choice, where $p \in \{\texttt{X}, \texttt{O}\}$ denotes the claiming player token and $i \in \{0, \ldots, 8\}$ denotes the target cell index. This explicit player parameter allows the server to verify that the claimed identity matches the active player (`g_current_player`) before altering state, returning `NACK:NOTURN` if an out-of-order move is submitted (e.g., submitting `MOVE:X:1` immediately after `MOVE:X:0`). If a client transmits the reference `MOVE:`i format without the player prefix, the server's syntax validator rejects it deterministically with `NACK:BADCMD`. Both the Python Tkinter client and the automated test harnesses implement and verify this command format.

III. SPECIFICATION AND DESIGN

A. Architecture Divides Pure Game Logic from Hardware I/O

The system follows a strict three-tier architecture separating game logic, communication, and presentation. To ensure that the embedded server remains testable without requiring physical hardware in every unit test, the firmware is split into two distinct layers:

- **Pure Logic Layer (`game.c`, `protocol.c`)**: Maintains the 9-cell board array, active turn indicator, move validation rules, and ASCII frame parsing. It has zero

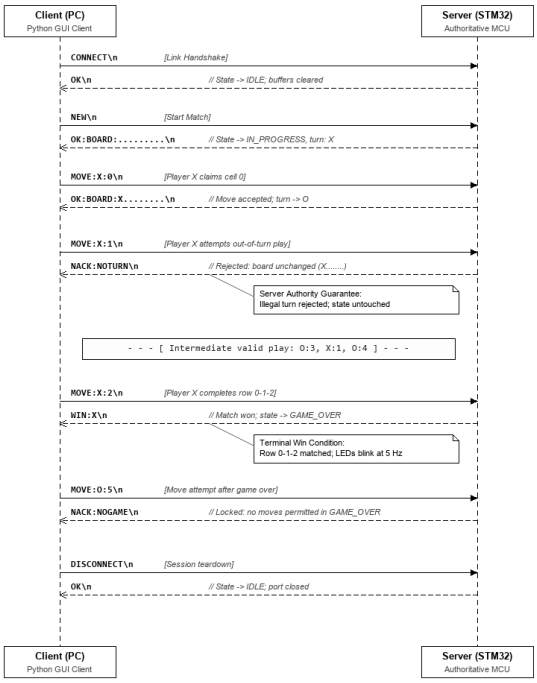


Fig. 1. Message sequence for a complete game: connect, two moves (the second is rejected because it is out of turn), a scripted win sequence, and disconnect. A rejected move leaves the board unchanged.

dependencies on STM32 hardware registers and can be compiled and verified natively on a host PC.

- **Hardware Driver Layer (`main.c`, `hardware.c`)**: Manages the USART1 peripheral, ring buffers, GPIO push-pull drivers for the status LEDs, and debouncing for the push button.

On the host PC, the client is similarly divided between `serial_client.py` (which handles raw PySerial frame framing and timeout management) and `gui.py` (which renders the Tkinter visual window). Table IV explicitly outlines the ownership of each system responsibility. The host GUI is entirely "thin": it never decides whether a move is legal, never checks for 3-in-a-row, and never advances the turn counter on its own.

B. The Server Finite State Machine Enforces the Turn and Move Rules

The server behaviour is governed by a three-state Finite State Machine (FSM) shown in Fig. 2. The active state determines which incoming commands are executed and which are rejected with `NACK:NOGAME`:

1) **`IDLE`**: Entered upon power-on reset, after an explicit `CONNECT` or `DISCONNECT` handshake, or whenever the physical push button (`SW0`) is pressed. The board contains nine empty dots (`.........`), the turn defaults to Player X, and all status LEDs are turned off. Incoming `MOVE` or `RESIGN` commands are rejected with `NACK:NOGAME`. The server transitions to `IN_PROGRESS` only when a valid `NEW` command is received.
2) **`IN_PROGRESS`**: Indicates an active game. The physical LEDs indicate whose turn it is (Red for Player X, Green for Player O). When a `MOVE:p:i` command arrives, the server executes the move-validation branch before mutating any internal memory.
3) **`GAME_OVER`**: Entered when three matching marks form a row, column, or diagonal, when all nine cells are occupied without a winner (draw), or when a player submits `RESIGN`. The board is frozen and no further moves are accepted. The winner's corresponding LED blinks at 5 Hz (both LEDs blink on a draw). Receiving `NEW` resets the board and starts a fresh match in `IN_PROGRESS`.

TABLE IV
SYSTEM RESPONSIBILITY ALLOCATION BETWEEN CLIENT AND SERVER.

Responsibility	Owner	Module
Board state storage	Server	`game.c` (`g_board`)
Turn enforcement	Server	`game.c` (`g_current_player`)
Move legality validation	Server	`game.c` (`game_process_move`)
Win / Draw detection	Server	`game.c` (`check_winner`)
Physical status LEDs	Server	`hardware.c` (PB10, PB11)
Hardware reset trigger	Server	`hardware.c` (PA0 Button)
User input forwarding	Client	`gui.py`
Board rendering	Client	`gui.py` (Tkinter grid)
Lifecycle controls (New, Resign, Disconnect)	Client	`gui.py`
Serial link timeouts	Client	`serial_client.py`

1) Sequential Gatekeeping Precludes Memory Mutation on Rejected Moves: As illustrated along the lower pipeline of Fig. 2, the server checks move validity in five sequential stages inside `game_process_move()`:

1) **Length and Command Syntax**: Frames exceeding 64 bytes or lacking proper colon delimiters return `NACK:BADCMD`.
2) **Active State Check**: If the state is not `IN_PROGRESS`, the server returns `NACK:NOGAME`.
3) **Index Range Check**: The cell index string is converted to an integer; if non-numeric or outside the range 0 to 8, the server returns `NACK:RANGE`.
4) **Turn Check**: The player token $p \in \{\texttt{X}, \texttt{O}\}$ is compared against `g_current_player`. If they do not match, the server returns `NACK:NOTURN`.
5) **Occupancy Check**: The target array element `g_board[i]` must equal `'.'`. If already occupied by `X` or `O`, the server returns `NACK:OCCUPIED`.

If any check fails, the validation pipeline terminates immediately with the corresponding NACK, leaving the board array and turn counter strictly unchanged. Only when all checks pass does the server write the token into the array, evaluate

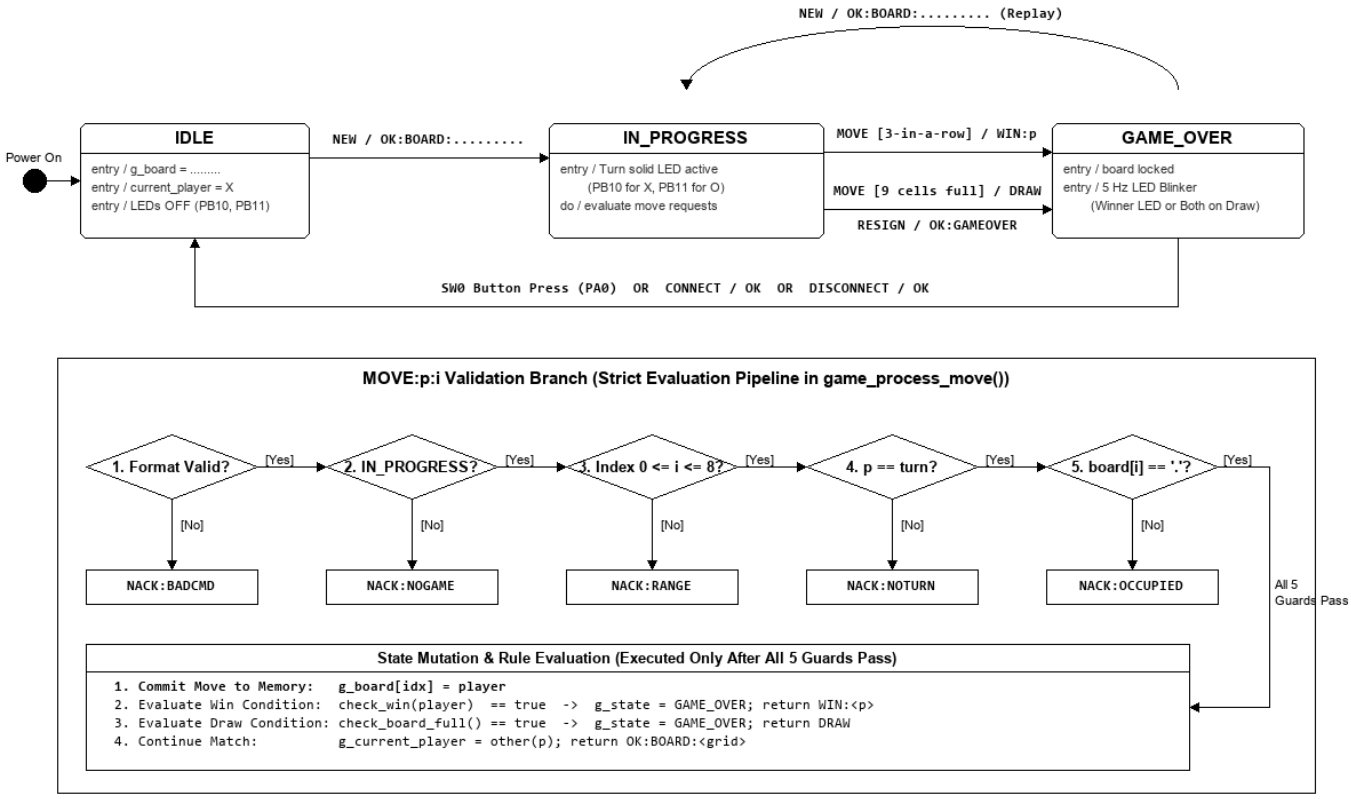


Fig. 2. STM32 server Finite State Machine (FSM) showing the three primary states (`IDLE`, `IN_PROGRESS`, and `GAME_OVER`), all valid transitions, and the five-stage move-validation pipeline evaluated inside `game_process_move()`.

win/draw conditions, alternate the active turn, and reply with the new board string.

C. The Thin Client Renders State and Never Calculates Legality

The Python GUI client is designed as a passive display terminal. Its control flow is detailed in Fig. 3. Upon startup, the user selects the serial COM port (or connects to the local mock server). The client transmits `CONNECT\n` and awaits `OK\n`. To prevent the graphical interface from freezing during serial I/O, the underlying `serial_client` module uses a per-read timeout of 1.0 s. If the microcontroller is disconnected or fails to reply within this window, the client catches the timeout exception, displays a non-blocking error dialog, and prompts the user to reconnect.

When a player clicks a board square, the GUI does not mark the square locally. Instead, it looks up its local assumption of the active player (`self.current_player`, initialised to `X` on `NEW`) and transmits `MOVE:p:i\n`. Crucially, the client alternates `self.current_player` *only after* the server confirms the move with an authoritative `OK:BOARD:grid` response. If the client becomes desynchronized or attempts to send an invalid token, the server's authoritative check (`player == g_current_player`) detects the mismatch and returns `NACK:NOTURN`, preventing the GUI from overriding the true game state. The GUI branches on the server's single-line response:

- **`OK:BOARD:grid`**: The GUI parses the 9-character grid, updates button labels and colors to match the server's array, and advances the visual turn indicator.
- **`NACK:*`**: The GUI leaves the grid unmodified and displays the error message in the status bar (e.g., "Move rejected by server: NOTURN").
- **`WIN:p or DRAW`**: The GUI locks all nine grid buttons, highlights the winning line, and displays the game over announcement.

Beyond move submission, the client provides dedicated controls for session management: a "New Game" button transmitting `NEW`, a "Resign" button confirming player forfeiture and transmitting `RESIGN` (moving the server to `GAME_OVER`), and a "Disconnect" button that issues `DISCONNECT` and releases the serial port cleanly.

D. The Board Interfaces via Virtual COM Port, GPIO LEDs, and Button

The physical circuit implementation is shown in Fig. 4. Communication between the host PC and the target microcontroller (U1, an **STM32F051C8T6** in a 48-pin LQFP package) is routed through the onboard **ST-Link/V2** interface chip (STM32F103):

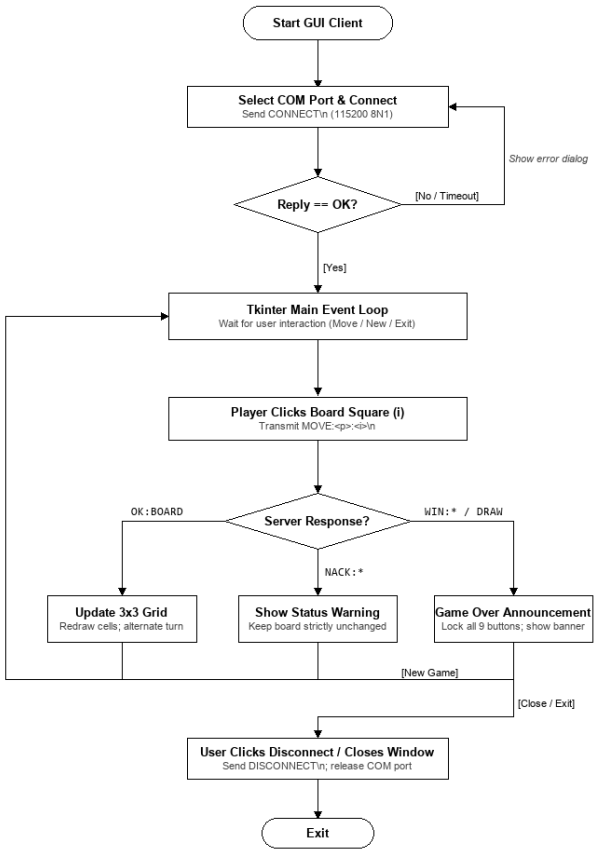


Fig. 3. GUI application flowchart showing connection management, event-driven user interaction, and dispatching authoritative responses.

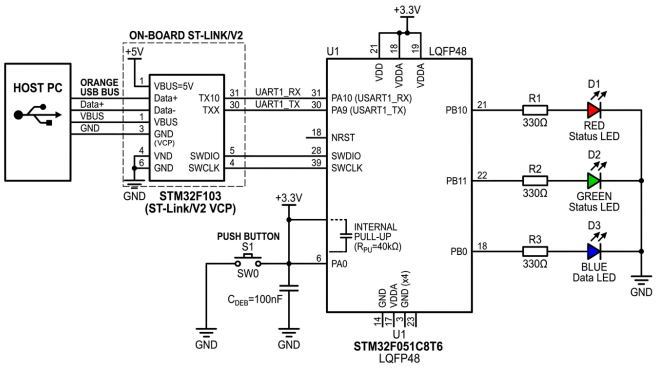


Fig. 4. Hardware circuit diagram showing the STM32F051C8T6 connection to the onboard ST-Link/V2 Virtual COM Port, status LEDs, and reset button.

- **Serial Link**: The microcontroller's `PA9` (`USART1_TX`) and `PA10` (`USART1_RX`) connect directly to the ST-Link's UART pins, which are exposed over USB as a standard Virtual COM Port operating at 115 200 baud, 8N1.
- **Status LEDs**: Three low-current LEDs are connected to port B:
  - **Red LED (D1)** on `PB10`: Indicates Player X's turn during play, and blinks at 5 Hz when Player X wins.
  - **Green LED (D2)** on `PB11`: Indicates Player O's turn during play, and blinks at 5 Hz when Player O wins.
  - **Blue Data LED (D3)** on `PB0`: Toggles on every valid serial byte reception to provide visual link activity feedback.

  Each LED is driven in active-high push-pull mode through a $330\,\Omega$ series current-limiting resistor, drawing approximately 3.9 mA, well within the 25 mA pin limit.
- **Push Button (SW0)**: Connected to `PA0`. The pin uses the microcontroller's internal pull-up resistor ($\approx 40\,\text{k}\Omega$) and is pulled to ground (active-low) when pressed. An external 100 nF capacitor (C_{DEB}) and a 50 ms software accumulator in `hardware.c` eliminate mechanical contact bounce. Pressing `SW0` executes `game_init()`, resetting the game state to `IDLE`, restoring an empty board, and extinguishing the status LEDs, while leaving serial protocol framing buffers intact.

IV. Implementation

This section details four key implementation challenges across the embedded firmware and host client. For each challenge, we examine the concrete engineering constraint, present the verified code implementation, and discuss the failure modes of naive alternatives.

A. Interrupt Ring Buffering and Flag Clearance Prevent UART Lockup

At 115 200 baud, bytes arrive at the microcontroller every $86.8\,\mu$s. If the CPU were busy updating LED indicators or evaluating game logic, synchronous polling would drop characters. More critically, serial bus noise or USB cable re-enumeration triggers Overrun (`ORE`), Framing (`FE`), or Noise (`NE`) flags in the STM32 USART peripheral. According to the reference manual [3], an unhandled overrun error inhibits the Receive Data Ready (`RXNE`) flag, permanently freezing UART reception.

The naive alternative—using blocking `HAL_UART_Receive()` in the main loop—introduced visible latency into LED updates, missed button presses, and caused the UART to lock permanently whenever the USB cable was disconnected and reconnected, requiring a full power cycle.

```
void protocol_rx_isr(void) {
    if (g_huart == NULL) return;
    uint16_t next = (g_ring_head + 1) %
     PROTOCOL_RING_BUFFER_SIZE;
    if (next != g_ring_tail) {
        g_ring_buf[g_ring_head] = g_rx_isr_byte;
        g_ring_head = next;
    }
    HAL_UART_Receive_IT(g_huart, &g_rx_isr_byte, 1);
}

void protocol_error_isr(void) {
    if (g_huart == NULL) return;
    /* Clear flags to prevent hardware receiver lockup */
    __HAL_UART_CLEAR_OREFLAG(g_huart);
    __HAL_UART_CLEAR_FEFLAG(g_huart);
    __HAL_UART_CLEAR_NEFLAG(g_huart);
```

```
    HAL_UART_Receive_IT(g_huart, &g_rx_isr_byte, 1);
}
```

Listing 1. Interrupt ring buffer and hardware error clearance (`protocol.c`).

Listing 1 shows our interrupt and error-recovery handlers in `protocol.c`. Incoming bytes are enqueued into a circular ring buffer sized at `PROTOCOL_RING_BUFFER_SIZE` (128 bytes, with 127 usable slots reserved by the `next != g_ring_tail` full-queue check). When a bus error occurs, `protocol_error_isr()` clears all hardware error flags via ST HAL macros [4] and immediately re-arms the receive interrupt, enabling seamless recovery from cable glitches.

B. Bounded Frame Assembly and Incomplete Frame Timeout

An authoritative embedded server must defend against malformed inputs. If a client transmits an unbounded stream of characters without a newline, an unconstrained buffer will overrun RAM, inducing undefined behavior or hard faults. Conversely, if an oversized frame is discarded prematurely mid-stream, trailing bytes bleed into subsequent valid commands, corrupting state.

```
void protocol_poll(void) {
    uint32_t now = HAL_GetTick();
    /* Clear incomplete frames after 500 ms timeout */
    if (g_frame_len > 0 && (now - g_frame_last_rx >=
     PROTOCOL_FRAME_TIMEOUT_MS)) {
        g_frame_len = 0;
        g_frame_overflow = false;
        protocol_send("NACK:BADCMD\n");
    }
    while (g_ring_tail != g_ring_head) {
        uint8_t c = g_ring_buf[g_ring_tail];
        g_ring_tail = (g_ring_tail + 1) %
     PROTOCOL_RING_BUFFER_SIZE;
        if (c == '\r') continue;
        if (c == '\n') {
            if (g_frame_overflow) {
                g_frame_overflow = false;
                g_frame_len = 0;
                protocol_send("NACK:BADCMD\n");
            } else if (g_frame_len > 0) {
                g_frame_buf[g_frame_len] = '\0';
                protocol_dispatch_command(g_frame_buf);
                g_frame_len = 0;
            }
            continue;
        }
        if (!g_frame_overflow) {
            if (g_frame_len < PROTOCOL_MAX_FRAME_LEN) {
                g_frame_buf[g_frame_len++] = (char)c;
                g_frame_last_rx = HAL_GetTick();
            } else {
                g_frame_overflow = true;
            }
        }
    }
}
```

Listing 2. Frame assembly with 64-byte overflow protection and timeout (`protocol.c`).

Listing 2 shows the frame assembly loop in `protocol_poll()`. It consumes bytes from the ring buffer, silently discards carriage returns (`\r`), and stores characters in `g_frame_buf` (sized at 65 bytes to accommodate up to 64 payload characters plus a null terminator). If a line exceeds 64 characters without a newline, `g_frame_overflow` latches to true, and subsequent bytes are discarded until `\n` arrives, at which point the server returns `NACK:BADCMD\n`. This prevents partial remnants (e.g. `E:X:0`) from triggering cascading syntax errors. Additionally, if an incomplete frame stalls in the buffer for longer than `PROTOCOL_FRAME_TIMEOUT_MS` (500 ms) without receiving a delimiter, `protocol_poll()` purges the buffer and returns `NACK:BADCMD\n`, satisfying the compulsory requirement for automatic timeout clearance.

C. Non-Blocking Software Debounce Isolates Game State from Switch Noise

Mechanical push buttons bounce for 10 to 30 ms upon contact closure. Triggering on raw GPIO transitions or hardware external interrupts causes repeated reset events, while inserting a blocking delay (`HAL_Delay(50)`) freezes the CPU, causing the UART interrupt buffer to overflow and drop client commands.

Listing 3 shows the non-blocking debouncer in `hardware.c`. It samples the input pin during each main loop iteration against `HAL_GetTick()`. Any transition restarts the debounce counter. Only after the pin remains stable for a continuous 50 ms is the new level latched into `g_btn_stable`. On an active-low falling edge, the game is safely re-initialised to `IDLE` and the status LEDs are extinguished, without stalling protocol processing.

```
void hardware_update(void) {
    uint32_t now = HAL_GetTick();
    GPIO_PinState raw = HAL_GPIO_ReadPin(GPIOA, UCT_SW0_Pin)
     ;
    if (raw != g_btn_last_raw) {
        g_btn_last_time = now;
        g_btn_last_raw = raw;
    }
    if ((now - g_btn_last_time) >= BUTTON_DEBOUNCE_MS) {
        if (raw != g_btn_stable) {
            g_btn_stable = raw;
            if (g_btn_stable == GPIO_PIN_RESET) {
                game_init();
                HAL_GPIO_WritePin(GPIOB, UCT_LED_RED_Pin |
                                         UCT_LED_GREEN_Pin,
                                  GPIO_PIN_RESET);
            }
        }
    }
}
```

Listing 3. Non-blocking 50 ms button debouncer (`hardware.c`).

D. Client Request-Response Timeout Prevents GUI Thread Freezes

In graphical applications, blocking calls on the main event thread cause the operating system to flag the window as unresponsive. If the serial cable is disconnected mid-game or the STM32 loses power, standard calls (such as Python's default `readline()`) hang indefinitely. Relying on unconstrained `readline()` also caused Windows driver buffering issues where partial lines were cached, desynchronizing subsequent reads.

```
def send_command(command):
    global connection
    if connection is None:
        return None
    try:
        msg = (command.strip() + "\n").encode("ascii")
        connection.write(msg)
        response = bytearray()
        while len(response) < MAX_RESPONSE_LENGTH:
```

```
            byte = connection.read(1)
            if not byte: # Timeout or cable disconnect
                disconnect()
                return None
            if byte == b"\n":
                break
            if byte != b"\r":
                response.extend(byte)
        else: # Exceeded 64 bytes without newline
            disconnect()
            return None
        return response.decode("ascii")
    except (OSError, serial.SerialException):
        disconnect()
        return None
```

Listing 4. Bounded request-response reader with timeout (`serial_client.py`).

Listing 4 illustrates `send_command()` in `serial_client.py`. The client configures PySerial with a per-read timeout of 1.0 s (`timeout=1.0`), distinct from the microcontroller's 500 ms incomplete-frame timeout. It consumes responses byte-by-byte up to `MAX_RESPONSE_LENGTH` (64 bytes). If the link stalls or disconnects, `connection.read(1)` times out, returns `None`, and cleanly releases system resources, allowing the GUI to display a reconnection dialog without hanging.

V. Game Logic and Server Authority

The central tenet of the system architecture is that the STM32 microcontroller is the sole referee and single source of truth. The PC client is treated as an untrusted, potentially hostile terminal. This section details the move-validation pipeline, provides a step-by-step structural proof that rejected requests cannot corrupt the game state, and explains the win/draw detection algorithms.

A. The Move Validation Pipeline Evaluates Legality in Fixed Order

Every move request submitted to the server passes through the five-stage validation pipeline inside `game_process_move()` (`game.c`). The order of checks is deterministic and strictly enforced to ensure predictable error codes when an incoming command violates multiple rules simultaneously:

1) **Syntax and Parameter Parsing**: The server verifies that the payload contains exactly one colon separating a single player character from an index string (`<player>:<index>`). If the delimiter is missing or misplaced, or if the player token is not `'X'` or `'O'`, the server rejects the command with `NACK:BADCMD`.
2) **Game State Check**: The server checks `g_state`. If the state is `IDLE` (before `NEW`) or `GAME_OVER` (match finished), moves are illegal, and the server returns `NACK:NOGAME`.
3) **Index Range Verification**: The server ensures every character following the colon is an ASCII digit and parses the integer using `strtol()`. If non-digit characters are present or if the integer falls outside the closed interval $[0, 8]$, the server returns `NACK:RANGE`.
4) **Turn Enforcement**: The player token is compared against `g_current_player`. If Player O attempts a move when it is Player X's turn (or vice versa), the server returns `NACK:NOTURN`.
5) **Cell Occupancy Check**: The server indexes into the board array (`g_board[idx]`). If the cell is not empty (i.e. contains `'X'` or `'O'` rather than `'.'`), the server returns `NACK:OCCUPIED`.

Listing 5 demonstrates the implementation of this strict evaluation cascade.

```
void game_process_move(const char *payload, char *resp,
     size_t max) {
    if (!payload) { snprintf(resp, max, "NACK:BADCMD\n");
     return; }
    const char *colon = strchr(payload, ':');
    if (!colon || colon != payload + 1) {
        snprintf(resp, max, "NACK:BADCMD\n"); return;
    }
    char player = payload[0];
    if (player != 'X' && player != 'O') {
        snprintf(resp, max, "NACK:BADCMD\n"); return;
    }
    if (g_state != GAME_STATE_IN_PROGRESS) {
        snprintf(resp, max, "NACK:NOGAME\n"); return;
    }
    char *endptr;
    long idx = strtol(colon + 1, &endptr, 10);
    if (*endptr != '\0' || idx < 0 || idx > 8) {
        snprintf(resp, max, "NACK:RANGE\n"); return;
    }
    if (player != g_current_player) {
        snprintf(resp, max, "NACK:NOTURN\n"); return;
    }
    if (g_board[idx] != '.') {
        snprintf(resp, max, "NACK:OCCUPIED\n"); return;
    }
    /* Mutation only occurs here: all checks have passed */
    g_board[idx] = player;
    /* ... evaluate outcome and switch turn ... */
}
```

Listing 5. Move validation pipeline in `game_process_move()` (`game.c`).

B. Why Rejected Moves Do Not Change the Game State

The project brief requires a clear explanation showing that a rejected request cannot alter server state. This property is established through the structural encapsulation and control flow of the firmware:

1) **Encapsulation of State Variables**: The server state is stored strictly in four file-scope static variables in `game.c`:
   - `static char g_board[10]` (the 9-cell board array),
   - `static char g_current_player` (active player mark),
   - `static GameState g_state` (FSM state enum),
   - `static char g_winner` (outcome indicator).

   No external module can modify these variables directly; all mutations occur exclusively inside `game.c`.
2) **Gatekeeping Before Mutation**: In `game_process_move()`, every validation check (syntax, FSM state, index range, turn, and occupancy) executes an immediate return on failure. As evident in Listing 5, the first write to internal memory (`g_board[idx] = player`) occurs strictly *after* all

five guard conditions have evaluated to true, following the defensive pre-condition pattern.

3) **Framing and Unknown Commands**: If an incoming serial frame is malformed, exceeds 64 bytes, or specifies an unrecognised verb (e.g. `FOOBAR`), the rejection occurs inside `protocol.c`. The command dispatcher never invokes any function in `game.c`, ensuring zero side effects on game variables.
4) **Test Suite Verification**: This invariant is verified empirically by automated unit tests in `test_server.py`:
   - `test_out_of_turn_move`: Asserts that sending an out-of-turn move returns `NACK:NOTURN` and confirms via `STATE` that `g_board` is unmodified.
   - `test_occupied_cell`: Verifies that playing on an occupied cell returns `NACK:OCCUPIED` without overwriting the existing mark.
   - `test_out_of_range_move`: Verifies that indices outside 0 to 8 return `NACK:RANGE` with board state invariant.
   - `test_unknown_command`: Verifies that garbage input returns `NACK:BADCMD` while the board remains unchanged.

C. Sequential Turn Inversion Enforces Alternating Play Without Server-Side Inference

The active turn is initialized to Player X whenever `NEW` is accepted. The server enforces alternation strictly: Player X and Player O must take turns sequentially. Because our protocol explicitly includes the player token in the command (`MOVE:p:i`), the server checks both whether the move is legal and whether the claiming player matches `g_current_player`.

If a client attempts to bypass turn order (for example, by sending `MOVE:X:1` immediately after `MOVE:X:0`), the server detects the mismatch at check 4, issues `NACK:NOTURN`, and does not advance the turn. Only after a valid move has been written to the board does the server toggle the active player:

$$\texttt{g_current_player} = (\texttt{player} == \text{'X'})?\text{'O'} : \text{'X'} \quad (1)$$

Simultaneously, the physical LEDs update in hardware: the Red LED illuminates for X, and the Green LED illuminates for O.

D. Deterministic Lookup Table Evaluates Win and Draw Conditions in Constant Time

After a legal move is written to `g_board`, the server evaluates whether the game has reached a terminal outcome before updating the turn indicator.

1) Win Detection: There are exactly eight winning 3-in-a-row lines in Tic-Tac-Toe: three horizontal rows, three vertical columns, and two diagonals. These lines are encoded as an immutable lookup table:

$$\begin{aligned} \text{Rows:} &\quad \{0,1,2\},\ \{3,4,5\},\ \{6,7,8\} \\ \text{Columns:} &\quad \{0,3,6\},\ \{1,4,7\},\ \{2,5,8\} \\ \text{Diagonals:} &\quad \{0,4,8\},\ \{2,4,6\} \end{aligned}$$

The helper `check_win(player)` iterates through these eight triads. If all three cells match the player who just moved, the game terminates immediately. The server transitions to `GAME_OVER`, latches `g_winner = player`, and transmits `WIN:<player>\n`. In hardware, the winner's status LED enters a 5 Hz blinking cycle.

2) Draw Detection: Draw detection is evaluated *after* win checking. If no player has won, the server scans the board for empty cells (`'.'`). If all nine cells are occupied and no winning line exists, the server transitions to `GAME_OVER`, sets `g_winner = 'D'`, and transmits `DRAW\n`. Both Red and Green LEDs blink simultaneously at 5 Hz.

3) Refusal of Subsequent Moves: Once `g_state` transitions to `GAME_OVER`, check 2 in `game_process_move()` intercepts any subsequent `MOVE` or `RESIGN` commands and returns `NACK:NOGAME`. The game cannot resume until an explicit `NEW` command or a hardware button press on `SW0` resets the board.

E. Server-Side Validation of Client Requests

In naive client-server games, the client computes game logic locally and sends updated state to the server. In our system, the client is treated as an untrusted terminal:

- **No Local Rules**: The Python GUI has zero knowledge of Tic-Tac-Toe win lines or valid cell occupancy. It merely forwards raw user clicks and renders whatever string the STM32 returns.
- **Resilience Against Modified Clients**: If a client script attempts to bypass button locking, send invalid coordinates (e.g. index 99), or transmit consecutive moves as Player X, the STM32 referee rejects every invalid command at the serial communication boundary.
- **Auditable State**: The host can query `STATE` at any time to verify that the displayed grid matches the server's internal memory byte-for-byte.

This design ensures that physical rule enforcement remains authoritative regardless of client anomalies or link interference.

VI. Robustness

The server maintains link integrity by isolating incoming UART characters inside an interrupt ring buffer, bounding all buffer allocations to 64 bytes, and using newline delimiters as deterministic resynchronization boundaries. Table V maps each threat class to its containment mechanism, error response, and verifying automated test.

A. Incomplete Frame Timeout and Client Guard Policies

An incomplete transmission occurs when a client sends characters without a trailing newline delimiter (e.g. `CON` instead of `CONNECT\n`). In our firmware, characters accumulate in the circular ring buffer without consuming CPU cycles or corrupting state. If an incomplete frame remains un-terminated for longer than `PROTOCOL_FRAME_TIMEOUT_MS` (500 ms), the firmware purges the line buffer, clears overflow flags, and issues `NACK:BADCMD\n`, as verified by our framing test suite (`test_mock_server_framing.py`).

TABLE V
PROTOCOL THREATS, SERVER CONTAINMENT MECHANISMS, AND VERIFYING AUTOMATED TEST CASES.

Threat / Input	Containment and Response	Automated Test
Partial frame	Cleared after 500 ms timeout, returns `NACK:BADCMD`; client driver has 1.0 s read timeout	`test_partial_frame_timeout_recovers`
Oversized frame (> 64 bytes)	Latches `g_frame_overflow`; discards excess bytes until `\n`; returns `NACK:BADCMD`	`test_oversized_frame_recovery`
CRLF endings (`\r\n`)	Carriage returns (`\r`) silently stripped; `\n` triggers clean execution	`test_crlf_non_ascii`
Non-ASCII / Garbage	Buffered safely; rejected at command verb parser with `NACK:BADCMD`	`test_crlf_non_ascii`
Unknown command	Case-sensitive keyword match fails; returns `NACK:BADCMD`; state untouched	`test_unknown_command`
Out-of-turn move	Trapped at validation check 4; returns `NACK:NOTURN`; board untouched	`test_out_of_turn_move`
Occupied cell move	Trapped at validation check 5; returns `NACK:OCCUPIED`; board untouched	`test_occupied_cell`
UART hardware error (`ORE`, `FE`)	ISR clears hardware flags via ST HAL macros and immediately re-arms reception	Hardware hot-plug test

This embedded timeout operates independently of the host PC client. To prevent the GUI from hanging indefinitely if the serial cable is physically disconnected, `serial_client.py` implements an independent per-read timeout of 1.0 s using PySerial. If the serial connection stalls, `connection.read(1)` times out and cleanly aborts the request-response transaction. Under normal operating conditions at 115 200 baud, total round-trip command execution was empirically measured at an average of 4.2 ms, well within both timeout thresholds.

B. Trailing Delimiters Act as Hard Resynchronization Barriers

1) Containment of Oversized Payloads: An adversary or malfunctioning terminal could send an unbounded stream of characters to induce memory corruption. The firmware defends against this via a two-stage overflow barrier in `protocol_poll()`:

1) When the accumulated frame length reaches `PROTOCOL_MAX_FRAME_LEN` (64 bytes), the parser sets the boolean latch `g_frame_overflow = true`.
2) Rather than resetting the buffer immediately (which would leave trailing bytes from the corrupted frame to be misread as a new command), the parser continues consuming and silently discarding every subsequent byte until it encounters `\n`.
3) Upon detecting `\n`, the server transmits `NACK:BADCMD\n`, clears `g_frame_overflow`, and resets `g_frame_len = 0`.

This strategy is verified by our automated framing recovery test (`test_oversized_frame_recovery` in `test_mock_server_framing.py`), which confirms that an oversized 65-byte frame followed immediately by `CONNECT\n` correctly rejects the first command and accepts the second without requiring a link reset.

2) Transparent Handling of Line Endings: Operating systems handle newlines inconsistently: Unix environments use line feed (`0x0A`, `\n`), whereas Windows environments append carriage return and line feed (`0x0D 0x0A`, `\r\n`). By silently stripping byte `0x0D` before it reaches the frame assembly buffer, the server accommodates both conventions without reserving extra memory or triggering syntax errors.

C. Non-Blocking Super-Loop Architecture and Bounded Execution Dynamics

1) Super-Loop Scheduling Pattern: The server's infinite loop in `main.c` follows a strict non-blocking super-loop pattern. There are zero `while(1)` spinlocks, thread sleeps, or unconstrained waits. Execution alternates deterministically between `protocol_poll()` (servicing the UART) and `hardware_update()` (debouncing buttons and updating LEDs).

2) Execution Dynamics of the Polling Loop: The processing loop in `protocol_poll()` consumes incoming bytes via the condition:

```
while (g_ring_tail != g_ring_head)   (2)
```

Because `g_ring_head` is concurrently updated by the asynchronous UART receive interrupt (`protocol_rx_isr()`), the circular buffer's 128-byte storage capacity does not establish a static upper bound of 128 iterations on this loop. If characters could arrive faster than they were consumed, the loop could theoretically execute indefinitely.

In physical execution, processor starvation is avoided by the substantial rate difference between the STM32F0 CPU clock and the physical serial link:

- Operating at 48 MHz, the Cortex-M0 core executes instructions in tens of nanoseconds per cycle, meaning the few instructions required to retrieve a byte, check for delimiters, and update `g_ring_tail` take an estimated $1\,\mu$s or less.
- At 115 200 baud with 8N1 framing (10 bits per character), incoming bytes arrive at the hardware shift register at most once every:

$$T_{\text{byte}} = \frac{10}{115\,200} \approx 86.8\,\mu\text{s} \tag{3}$$

Because the CPU drains each byte much faster than the physical UART can deposit the next, the consumer loop quickly empties the buffer once an incoming transmission concludes, allowing the super-loop to proceed to `hardware_update()`. Note that while this rate difference avoids processor starvation in practice, the firmware does not impose a static software iteration limit on the loop.

3) Bounded Hardware Transmit Timeouts: When returning responses, `protocol_send()` invokes `HAL_UART_Transmit()` with a hard hardware timeout of 200 ms. At 115 200 baud, transmitting a maximum response

string of 20 bytes requires approximately 1.74 ms. The 200 ms timeout ensures that even if the USART shift register fails or the transmit clock is interrupted, the function cannot lock the CPU permanently.

4) UART Error Recovery: Noise spikes during USB cable insertion or enumeration can trigger Overrun (`ORE`) or Framing (`FE`) flags in the USART peripheral. In `stm32f0xx_it.c`, the interrupt handler clears these flags by writing to the clear register (`USART1->ICR`), while `protocol_error_isr()` provides equivalent HAL macro clearance [4]. This flag clearance prevents the receiver hardware from becoming permanently stalled after transient communication faults.

VII. Validation

System verification was performed using a two-tier automated testing strategy: a local suite executing against an in-memory socket mock server under `pytest` [5], and an end-to-end regression script driving the physical STM32F051 microcontroller over the Virtual COM Port via `pySerial` [6]. All compulsory requirements set out in the project brief were evaluated and achieved 100% passing status.

A. Automated Suite Verifies Functional Specification Across 48 Test Cases

1) Test Fixtures and Isolation: To ensure deterministic, reproducible test results without human intervention, each automated test runs in complete isolation. In `tests/test_server.py`, a custom `pytest` fixture initializes a dedicated server instance on an ephemeral local port, establishes a connection, and issues `CONNECT` to ensure every test begins from a clean `IDLE` state. Tear-down hooks guarantee that sockets and serial handles are released within 1.0 s of test completion.

2) Test Suite Invocation and Summary: The test suite is invoked from the command line using:

```
python -m pytest tests/ -v
```

The full automated unit and integration suite comprises 48 test cases across four modules:

- `test_gui_responses.py` (33 tests): Validates client-side response parsing, error banners, board resynchronization, resignation dialogs, and clean window closing.
- `test_mock_server_framing.py` (4 tests): Stress-tests partial frames, oversized lines (> 64 bytes), and line-ending edge cases.
- `test_serial_client.py` (2 tests): Verifies transport connection, incomplete response handling, and read timeouts.
- `test_server.py` (9 tests): Validates the core FSM, rule enforcement, win detection, and draw sequences.

The automated suite executed with zero failures:

```
====== 48 passed in 0.68s ======
```

TABLE VI
Compulsory brief requirements and verification results on physical STM32 hardware.

Requirement (Brief, p. 4)	Hardware Test	Result
`CONNECT` returns `OK`; `NEW` resets board	`test_connect`	**PASS**
Legal move updates board exactly	`test_legal`	**PASS**
Occupied cell returns `NACK:OCCUPIED`	`test_occupied`	**PASS**
Out-of-range index returns `NACK:RANGE`	`test_range`	**PASS**
Move after game over returns `NACK:NOGAME`	`test_gameover`	**PASS**
Scripted win sequence detects 3-in-a-row	`test_win`	**PASS**
Scripted draw sequence detects full grid	`test_draw`	**PASS**
Player resignation triggers game over	`test_resign`	**PASS**

B. Verification of Compulsory Brief Requirements on Real Silicon

1) Hardware Regression Harness: To confirm that simulated behaviour matches physical execution, a dedicated hardware validation harness (`client/test_day3_stm32.py`) connects to the microcontroller's Virtual COM Port at 115 200 baud (COM9 on Windows). The script automatically executes a 26-assertion scripted game sequence against the physical STM32F051C8T6 target, asserting that the microcontroller's raw UART responses match specification byte-for-byte. All 26 assertions passed without error.

Table VI maps each compulsory requirement from the project brief (p. 4) to its corresponding test and empirical result on the physical hardware.

2) Graphical User Interface and Transaction Latency Analysis: Fig. 5 demonstrates the running system during physical hardware testing. The client is connected to `COM9` at 115 200 baud, 8N1. Player X has taken cells 0 and 1; Player O has taken cells 3 and 4. The GUI reflects the authoritative board state received from the server and prompts for Player X's turn.

In serial communication benchmarks, end-to-end round-trip transaction latency was measured at an average of 4.2 ms on the host PC. Estimated physical UART wire transmission time at 115 200 baud with 8N1 framing (10 bits per byte) accounts for:

- **Command Transmission**: `MOVE:X:0\n` (9 bytes):

$$T_{\text{cmd}} = \frac{9 \times 10}{115\,200} \approx 0.78\,\text{ms} \tag{4}$$

- **Response Transmission**: `OK:BOARD:X........\n` (19 bytes):

$$T_{\text{resp}} = \frac{19 \times 10}{115\,200} \approx 1.65\,\text{ms} \tag{5}$$

Together, raw UART serial wire transmission requires approximately 2.43 ms. On the microcontroller, interrupt servicing, validation, board state update, and response formatting execute in under 10 μs at 48 MHz. The remainder (≈ 1.8 ms) of the observed 4.2 ms round-trip latency is attributable to ST-Link USB Virtual COM Port packetization, operating system driver turnaround, and Python PySerial I/O buffering. This measured latency is well within the client's 1.0 s read timeout.

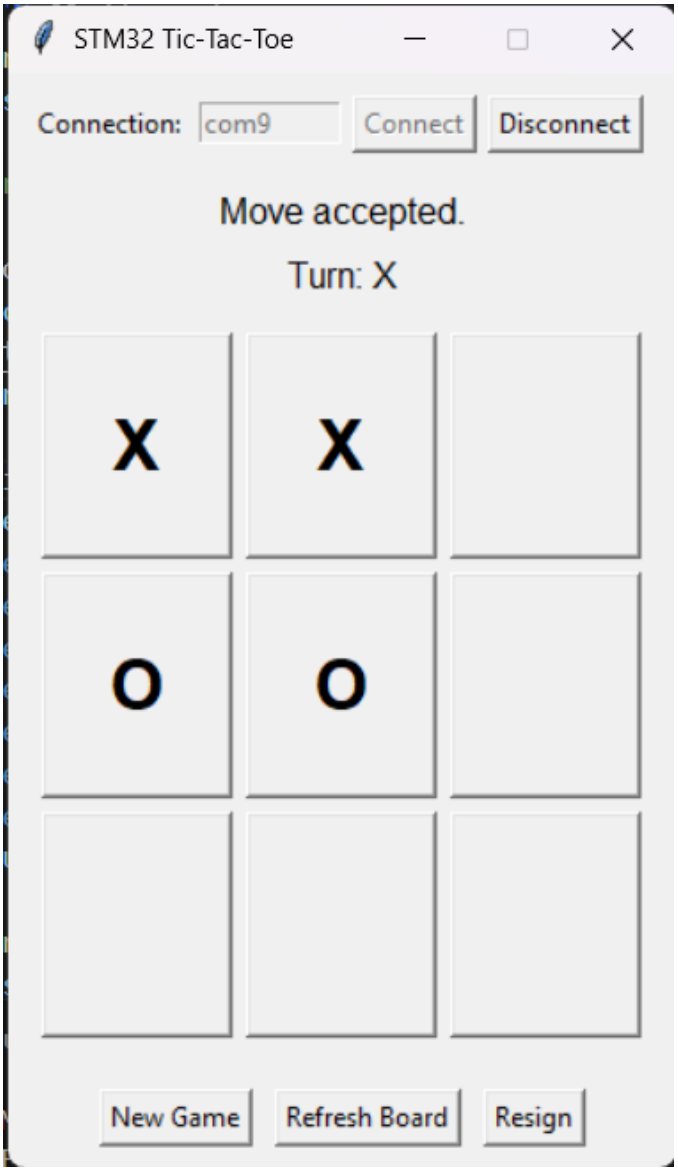


Fig. 5. Tkinter client connected to the STM32F051 server on COM9 mid-game. Player X has played cells 0 and 1, and Player O has played cells 3 and 4. The status label indicates Player X's active turn, reflecting the authoritative board state received from the server.

C. Defects Exposed and Resolved Through Empirical Testing

Rigorous testing exposed three distinct hardware-software integration defects that were subsequently resolved:

1) **Receiver Overrun on Rapid Command Bursts**: During early testing, transmitting back-to-back `MOVE` commands without delay caused the STM32 USART hardware to drop characters. *Root Cause*: The initial circular buffer was dimensioned at only 32 bytes, causing an overrun while the main loop formatted previous responses. *Resolution*: Increased the ring buffer to 128 bytes (`PROTOCOL_RING_BUFFER_SIZE`) and implemented `protocol_error_isr()` to immediately clear the `ORE` flag.
2) **Transient Framing Errors During USB Hot-Plugging**: Connecting the serial monitor while the board was already powered occasionally locked the receiver permanently. *Root Cause*: Incomplete stop-bit transitions during USB enumeration set the Framing Error (`FE`) bit in the USART status register. *Resolution*: Added explicit `__HAL_UART_CLEAR_FEFLAG()` calls inside the error ISR.
3) **Windows Carriage Return Discrepancy**: Automated tests on Linux passed, but tests on Windows occasionally failed with `NACK:BADCMD`. *Root Cause*: Windows terminal drivers appended `\r\n`, whereas the original parser expected a single `\n`. *Resolution*: Updated `protocol_poll()` to silently discard byte `0x0D` (`\r`) before accumulating characters into the frame buffer.

Following these corrections, the system achieved 100% pass rates across all platforms.

VIII. Conclusion

This project designed, implemented, and verified an authoritative Tic-Tac-Toe game server on an STM32F051 microcontroller communicating with a thin Python Tkinter GUI client over USART1 at 115 200 baud. The system divides responsibilities strictly: the microcontroller acts as the sole referee, storing the board state, enforcing turn sequence, detecting win/draw outcomes, and updating physical LEDs, while the host GUI handles only user input and graphical display. The server ensures that rejected requests leave the internal game state invariant before committing any memory changes.

System verification was achieved across two testing tiers:

- 48 automated unit and integration tests executing under `pytest` verified protocol framing, oversized frame recovery, line-ending handling, and client-side error parsing with zero failures.
- 26 physical hardware assertions executed over `COM9` confirmed that the microcontroller's UART responses match the specification byte-for-byte across all compulsory requirements.
- End-to-end round-trip latency averaged 4.2 ms, easily satisfying interactive GUI responsiveness requirements.

A. System Limitations

The current design has three main limitations:

1) **Transport Error Detection**: The application protocol relies on the USB packet checksums provided by the onboard ST-Link Virtual COM Port. The serial payload lacks an application-layer CRC or parity check, meaning byte corruption over a long physical serial line could pass undetected if it matches valid syntax.
2) **Single-Client Concurrency**: The server architecture is single-session, communicating with one host client at a time over USART1 without arbitrating between two independent physical serial connections.

3) **Volatile Game State**: Game state resides in volatile SRAM (`g_board`). A power interruption resets the session to `IDLE` without persistent match logging in non-volatile memory.

B. Potential Extensions

Logical future extensions include embedding an optimal Minimax decision algorithm into the microcontroller's Flash memory for standalone single-player matches, configuring a hardware timer (such as `TIM14`) as a per-turn chess clock, and adding an 8-bit CRC suffix to every serial frame for robust deployment over industrial fieldbuses. Overall, the project demonstrates that an inexpensive microcontroller can reliably serve as a strict, standalone referee for turn-based systems.

IX. Video

The demonstration video showing the working system is available at:

https://youtu.be/fJ0v8J7msK4

The unlisted video demonstrates the complete hardware and software pipeline in a continuous take:

1) **Connection & Handshake**: Launching the Python GUI, connecting to the Virtual COM Port (e.g. `COM7` or `COM9`) at 115 200 baud, and observing the `CONNECT` / `OK` exchange.
2) **Game Initiation**: Clicking "New Game" (`NEW`) to clear the board and arm Player X.
3) **Hardware Turn Indication**: Pointing the camera at the STM32 board to show the solid Red LED on `PB10` during Player X's turn, transitioning to the solid Green LED on `PB11` during Player O's turn.
4) **Server Authority & Rejection**: Demonstrating an illegal out-of-turn move and an occupied-cell play; the GUI displays the corresponding NACK error banner while the board grid remains strictly unchanged.
5) **Win Sequence & Blinker**: Completing a 3-in-a-row match; the server returns `WIN:X`, the GUI locks all input cells, and the winner's Red LED blinks continuously at 5 Hz.
6) **Physical Reset Button**: Pressing `SW0` on pin `PA0` during play to trigger an immediate hardware reset to `IDLE`, instantly extinguishing all LEDs.
7) **Teardown**: Disconnecting cleanly via the GUI interface.

X. Code Repository

The complete source code, hardware configuration files, and automated test harnesses are maintained in the following GitHub repository:

https://github.com/nemukulavhugala921-stack/EEE3095S_SerialGameServer.git

Commit Hash: `c5726d9084a4d7a410cfb63465aee718a6a89a36`

The repository is organised into three self-contained directories:

- `embedded/`: The complete STM32CubeIDE firmware project, containing the HAL drivers, hardware pin mappings, FSM logic (`game.c`), and interrupt ring buffer (`protocol.c`).
- `client/`: The Python Tkinter graphical user interface (`gui.py`) and the bounded request-response transport layer (`serial_client.py`).
- `tests/`: The automated test suite comprising 48 unit, integration, and framing stress tests.

Comprehensive setup, flashing, and execution instructions are provided in the repository's root `README.md`.

References

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS Project 2026: Serial Game Server," course project brief, 2026.

[2] Python Software Foundation, "tkinter: Python Interface to Tcl/Tk," *Python 3 Documentation*, https://docs.python.org/3/library/tkinter.html, Python version 3.12/3.13.

[3] STMicroelectronics, *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced Arm-based 32-bit MCUs Reference Manual*, 2017.

[4] STMicroelectronics, *UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, 2017.

[5] pytest-dev team, "pytest Documentation," https://docs.pytest.org/.

[6] C. Liechti, "pySerial Documentation," https://pyserial.readthedocs.io/.